OrderShard Backend API Documentation

This document describes the API endpoints, request/response formats, and inner architectural logic for the OrderShard backend ingestion pipeline.

Tech Stack & Architecture Overview

- **Runtime:** Node.js (Express)
- **Database:** PostgreSQL (Prisma ORM)
- **Partitioning:** Declarative Hashing (4 shards)
- **Queue:** BullMQ + Redis
- **Storage:** GCS (Google Cloud Storage)

API Endpoints

1. Ingest Orders File (CSV)

Uploads a CSV file of orders, saves it to GCS, and queues it for background worker processing.

POST `/upload-orders` **(Alias:** `/api/orders/upload` **)**

- **Content-Type:** `multipart/form-data`
- **Request Body:** `file` (Required): The CSV file containing order records.

cURL Command:

```
curl -X POST -F "file=@/path/to/your/orders.csv" http://localhost:5000/upload-orders
```

Response (202 Accepted):

```
{
  "success": true,
  "message": "File uploaded successfully",
  "jobId": "12"
}
```

2. Get Ingestion Job Status

Queries the progress of an asynchronous CSV ingestion job using its unique BullMQ Job ID.

GET `/api/jobs/:jobId` **cURL Command:**

```
curl -X GET http://localhost:5000/api/jobs/12
```

Response (200 OK - In Progress):

```
{
  "success": true,
  "jobId": "12",
  "status": "active",
  "progress": {
    "progress": 45,
    "validCount": 4500,
    "invalidCount": 5,
    "totalRecords": 10000,
    "batches": 9
  }
}
```

3. Fetch Recent Orders

Retrieves the latest ingested orders from the PostgreSQL partitioned table.

GET `/orders`

- **Query Params:** `limit` (default 30), `customerId` (optional).

Response (200 OK):

```
{
  "success": true,
  "data": [
    {
      "id": "7f3a8b2c-9100-4b82-990a-28d82bd921f0",
      "customer": "cus_10428",
      "date": "2026-07-24 10:22",
      "amount": "142.90",
      "status": "completed",
      "shard": "shard-0"
    }
  ]
}
```


4. Fetch Shard Fleet Stats

Queries row count metrics directly from the PostgreSQL partition tables (orders_p0 to orders_p3).

GET `/orders/shards` **Response (200 OK):**


```
{
  "success": true,
  "data": [
    {
      "name": "shard-0",
      "host": "localhost:5432 (orders_p0)",
      "rows": "2,406",
      "load": 10,
      "status": "healthy"
    }
  ]
}
```


5. Fetch Single Order by ID

GET `/orders/:orderId`

Architectural Logic

Shard Resolution Formula

The shard is determined using the MD5 hashing of the `customer_id` modulo 4:

1. Compute MD5 checksum of `customer_id`.
2. Extract the first 8 characters and parse them as a hex integer.
3. Apply **modulo 4** to get the partition index (0, 1, 2, or 3).

CSV Row Validation Criteria

The streaming parser validates each row against the schema constraints:

- `customer_id`: Must be present.
- `order_date`: Must be a valid date format.
- `order_amount`: Must be a positive numeric value.
- `status`: Must be one of PENDING, PROCESSING, COMPLETED, CANCELLED.